Programozási nyelvek 1

Szathmáry László
Debreceni Egyetem
Informatikai Kar

5. előadás

- const, fordítás, hibakeresés, stílus
- sztringek

(utolsó módosítás: 2026. febr. 3.)

2025-2026, 2. félév

Tömb átadása paraméterként

- Amikor átadunk egy tömböt paraméterként, akkor érdemes átadni a tömb méretét, ill. magát a tömböt. Érdemes ezt a sorrendet követni, vagyis előbb a méretet adjuk át, aztán magát a tömböt.

- Amikor egy tömböt átadunk paraméterként, akkor MINDIG gondoljuk át, hogy a tömböt milyen céllal adjuk át.
  - Azt akarom, hogy a meghívott függvény módosítsa a tömbömet.
  - Csak át akarom adni a tömböt feldolgozásra, de nem szeretném, ha a függvény bármit is módosítana a tömbön.

Tömb átadása paraméterként

```c
#include <stdio.h>

void szoroz_10_zel(int n, int tomb[])
{
    for (int i = 0; i < n; ++i)
    {
        tomb[i] *= 10;    // tomb[i] = tomb[i] * 10;
    }
}

int main()
{
    int szamok[4] = { 1, 4, 8, 3 };
    int meret = 4;

    szoroz_10_zel(meret, szamok);
    printf("%d\n", szamok[0]);
    printf("%d\n", szamok[1]);
    printf("%d\n", szamok[2]);
    printf("%d\n", szamok[3]);

    return 0;
}
```

Tömb átadása paraméterként

```c
#include <stdio.h>

int sum(int n, const int tomb[])
{
    int osszeg = 0;
    for (int i = 0; i < n; ++i)
    {
        osszeg += tomb[i];
    }

    return osszeg;
}

int main()
{
    int szamok[4] = { 1, 4, 8, 3 };
    int meret = 4;

    int osszeg = sum(meret, szamok);
    printf("osszeg: %d\n", osszeg);
    //
    printf("%d\n", szamok[0]);
    printf("%d\n", szamok[1]);
    printf("%d\n", szamok[2]);
    printf("%d\n", szamok[3]);

    return 0;
}
```

const = *read-only*

A változó kezdőértéket kap az inicializáció során, majd a változó értéke nem változhat meg (*immutable*).

Ez mire jó?

Kapunk egy garanciát arra, hogy a változó értéke nem fog megváltozni.

A változó védve van a véletlen módosításoktól. Ha mégis módosítani szeretnénk, akkor fordítási hibát kapunk.

const

```c
#include <stdio.h>

int terulet(const int a, const int b)
{
    return a * b;
}

int main()
{
    const int a = 3;
    const int b = 2;

    const int t = terulet(a, b);
    printf("terület: %d\n", t);

    return 0;
}
```

Tetszőleges változó ellátható a *const* módosító jelzővel.

Vagyis nem csak a formális paraméterlistán használható!

Ha egy változó értékét szeretnénk megvédeni a módosításoktól, akkor használjuk bátran!

Tipp: miután megírtuk a programunkat, nézzük át ismét, s gondoljuk át, hogy miket lenne érdemes *const* -tá tenni. Mi az, aminek végleges az értéke, ami biztosan nem módosul (vagy legalábbis nem kellene módosulnia)?

Rust példa

```rust
fn terulet(a: i32, b: i32) -> i32
{
    return a * b;
}

fn main()
{
    let mut a: i32 = 3;
    let b: i32 = 2;

    a = 4;
    b = 5;    // Hiba! b immutable

    let t = terulet(a, b);
    println!("terület: {}", t);
}
```

Rust-ban a *read-only* mód az alapértelmezés.

Ha valamit módosíthatóvá akarunk tenni, akkor azt külön jeleznünk kell.

A modernebb programozási nyelvekben lévő jó ötleteket nyugodtan át lehet venni. Lehet őket „backportolni” C-be.

Kotlin példa

sum.kt

```
1   fun sum(x: Int, y: Int): Int
2   {
3       x = 5;    // Hiba! x read-only
4       return x + y
5   }
6
7   fun main()
8   {
9       println(sum(1, 2))
10  }
11
```

Kotlin-ban a formális paraméterek alapértelmezetten *read-only* módban vannak! A hívás pillanatában kezdőértéket kapnak, ami utána nem módosítható.

Ha mégis módosítani akarjuk valamelyik paramétert, akkor arról készítünk egy módosítható másolatot.

A modernebb programozási nyelvekben lévő jó ötleteket nyugodtan át lehet venni. Lehet őket „backportolni” C-be.

Fordítás

```c
#include <stdio.h>

int main()
{
    printf("hello world\n");

    return 0;
}
```

```
$ gcc hello.c

$ ./a.out
```

Leegyszerűsítve:

Fordítás

```c
#include "prog1.h"
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

```
$ gcc hello1.c prog1.c

$ ./a.out
```

Fordítás

A fordítás lépései részletesen:

- előfeldolgozás (*preprocessing*) [C forrás → C forrás]
- fordítás (*compiling*) [C forrás → tárgykód]
  - assembly kód generálása [C forrás → assembly forrás]
  - assembly kód fordítása [assembly forrás → tárgykód]
- kapcsolatszerkesztés (*linking*) [tárgykód(ok) → futtatható EXE]

Fordítás - előfeldolgozás

```
#include "prog1.h"
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - előfeldolgozás

```
#include "prog1.h"
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - előfeldolgozás

```
string get_string(const string prompt);
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - előfeldolgozás

```
string get_string(const string prompt);
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - előfeldolgozás

```
string get_string(const string prompt);
int printf(string format, ...);

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - előfeldolgozás

```
...
string get_string(const string prompt);
int printf(string format, ...);
...

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - fordítás

```
    .file   "hello1.c"
    .text
    .section    .rodata
.LC0:
    .string "Mi a neved?\n"
.LC1:
    .string "Hello %s!\n"
    .text
    .globl  main
    .type   main, @function
main:
.LFB0:
    .cfi_startproc
    pushq   %rbp
    .cfi_def_cfa_offset 16
    .cfi_offset 6, -16
    movq    %rsp, %rbp
    .cfi_def_cfa_register 6
    subq    $16, %rsp
    leaq    .LC0(%rip), %rdi
    call    get_string@PLT
    movq    %rax, -8(%rbp)
    movq    -8(%rbp), %rax
    movq    %rax, %rsi
    leaq    .LC1(%rip), %rdi
    movl    $0, %eax
    call    printf@PLT
    movl    $0, %eax
    leave
    .cfi_def_cfa 7, 8
    ret
    .cfi_endproc
.LFE0:
    .size   main, .-main
    .ident  "GCC: (Arch Linux 9.2.1+20200130-2) 9.2.1 20200130"
    .section    .note.GNU-stack,"",@progbits
```

C kód

↓

assembly kód

↓

tárgykód

Fordítás - fordítás

```
0001011111100011101100110001101001111
1010100011100001100010100110010011000
0
1100110101000000100011011101010100011
1
0000000011111001010110011000100100110
1
0100010101000000100000101101110101100
11
0110011001000010001011000100011010110
0
1100010001110100011011110101111010000
11
1111101100001101010110010101100001010
1
1110000010000100101101011000100111101
1
1100100110000000101110000010101001000
00
1111111000111001011101100111111010000
0
0101110100010010101010101100010010011
1
1101010111101110101101111010001101100
0
0101111010000010000011011101101111000
1
1001011010111111000011001110111011010
0
1100101000101011111110001011101100001
1
1010001100001101011100111111100010111
1
0010111001001001111100011110000000001
```

C kód

↓

assembly kód

↓

tárgykód

Házi feladat:

Egy ilyen szemléltető ábra generálása.

Fordítás - kapcsolatszerkesztés

```
#include "prog1.h"
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - kapcsolatszerkesztés

```
#include "prog1.h"
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - kapcsolatszerkesztés

```
#include "prog1.h"
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - kapcsolatszerkesztés

```
#include "prog1.h"
#include <stdio.h>

int main()
{
    string s = get_string("Mi a neved?\n");

    printf("Hello %s!\n", s);

    return 0;
}
```

Fordítás - kapcsolatszerkesztés

hello1.c prog1.c stdio.c

Fordítás - kapcsolatszerkesztés

tárgykódok

hello1.c	prog1.c	stdio.c
↓	↓	↓
0101000	1100101	0010100
1010010	0010100	1001001
1001001	0101001	0001111
001…	110…	001…

Fordítás - kapcsolatszerkesztés

tárgykódok

futtatható EXE

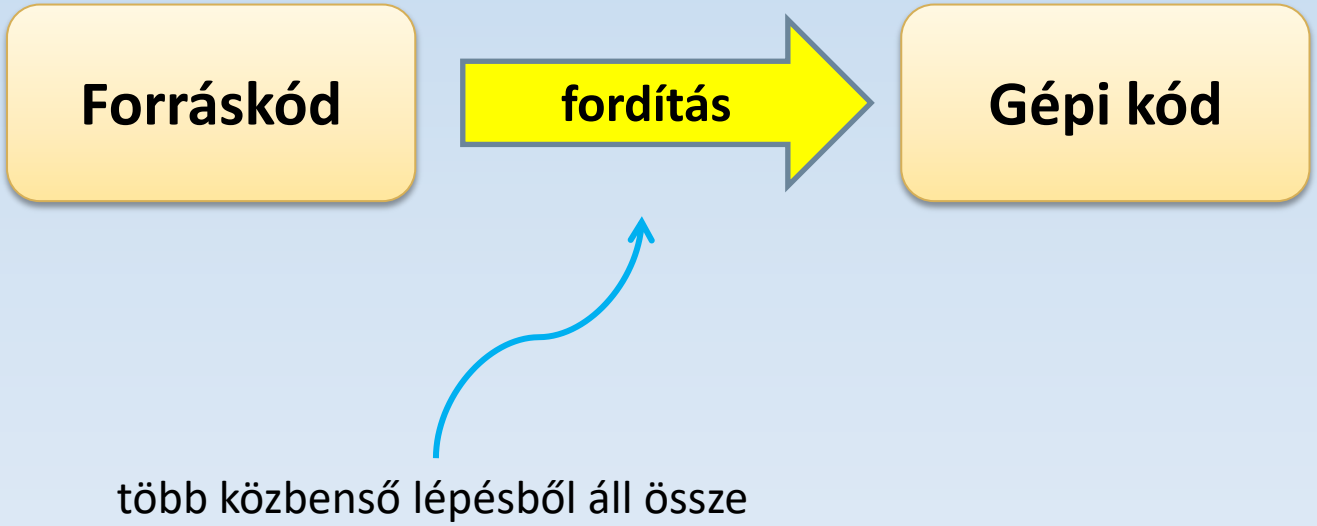
```
hello1.c          prog1.c          stdio.c
   ↓                ↓                ↓
0101000           1100101          0010100
1010010           0010100          1001001
1001001           0101001          0001111
001…              110…             001…
```

kapcsolat-
szerkesztés

```
…0100100100100010110101011
1101101010001001010101010 0
1001011101010100010001001 0
01010100100101 0…
```

Fordítás

Leegyszerűsítve:



Hibakeresés (debugging)

A **bug** a számítógépes **programhiba** elterjedt elnevezése.

Hibák esetén beszélhetünk

- szintaktikai, és
- szemantikai

hibákról.

Hibakeresés

Hibakeresési módszerek:

- `printf()` használata (legegyszerűbb módszer, debug információk kiíratása)
- debugger használata (ez lehet parancssoros vagy grafikus felülettel rendelkező)

Hibakeresés

bug2c.c - sources - Visual Studio Code

File Edit Selection View Go Debug Terminal Help

RUN AND DE...

bug0.c | bug1.c | bug2a.c | bug | tasks.json | launch.json

VARIABLES

Locals

i: 4

bug2c.c > main()

```c
#include <stdio.h>

// Írassuk ki a '*' karaktert 5-ször.
// Használjunk debugger-t.

int main()
{
    for (int i = 0; i <= 5; ++i)
    {
        printf("*\n");
    }

    return 0;
}
```

WATCH

CALL STACK PAUSED ON STEP

main() bug2c.c 10:1

OUTPUT TERMINAL DEBUG CONSOLE PROBLEMS 2

```
*
*
*
*
```

BREAKPOINTS

bug2c.c 8

master* 2 0 gcc - Build and debug active file (sources)

hibakeresés debugger használatával

Stílus

„A stílus maga az ember.”

Georges-Louis Leclerc de Buffon

Az, hogy megírok egy programot és az lefut helyesen, az egy dolog.

Törekedjünk arra, hogy maga a forráskód is **szép** legyen. Egy program elkészítése egy szellemi termék, egy mérnöki munka. Nem mindegy, hogy mit adunk ki a kezünkből!

```
1  #include <stdio.h>
2  int main(){
3    printf("hello\n");
4  return 0;}
5
```

```
1  #include <stdio.h>
2
3  int main()
4  {
5      printf("hello\n");
6
7      return 0;
8  }
```

Stílus

- gondoljunk magunkra
- gondoljunk másokra
- gondoljunk arra, hogy egy programot karban is kell tartani

Ha majd elmennek dolgozni, akkor valószínűleg a cégnél is lesz egy *style guide*, amit minden programozónak követnie kell.

Nekünk is van egy *style guide*-unk, a félév során ehhez próbáljuk tartani magunkat (link, github).

Stílus

linter: olyan szoftver, amely a forráskódot elemzi. Célja: hibák, bug-ok felderítése, stílusbeli problémák jelzése.

A style50 nevű stíluselemző telepítése:

```
$ pip3  # győződjünk meg róla, hogy ez fent van

$ pip3 install style50 --user -U
$ sudo apt install astyle     # ez is kell hozzá
```

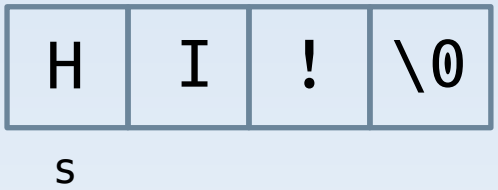
Használata:

```
$ style50 program.c
```

Sztringek

- Egy sztring valójában egy karaktereket tartalmazó tömb.
- C-ben egy karakter (`char`) mérete 1 byte.
- Ami különlegessé teszi: az utolsó karaktere után szerepel egy speciális karakter (`'\0'`), ami a sztring végét jelzi.

```
string s = "HI!";
```



Sztringek

- Egy sztring valójában egy karaktereket tartalmazó tömb.
- C-ben egy karakter (`char`) mérete 1 byte.
- Ami különlegessé teszi: az utolsó karaktere után szerepel egy speciális karakter (`'\0'`), ami a sztring végét jelzi.

```
string s = "HI!";      // a mem.-ban 4 byte-ot foglal
```

H	I	!	\0
s[0]	s[1]	s[2]	s[3]

Sztringek

```
string nevek[4];
nevek[0] = "Anna";
nevek[1] = "Bea";
nevek[2] = "Cecil";
nevek[3] = "David";
```

A nevek[0]	n	n	a	\0	B nevek[1]
e	a	\0	C nevek[2]	e	c
i	l	\0	D nevek[3]	a	v
i	d	\0			

Sztringek

0	NUL	16	DLE	32	SP	48	0	64	@	80	P	96	`	112	p
1	SOH	17	DC1	33	!	49	1	65	A	81	Q	97	a	113	q
2	STX	18	DC2	34	"	50	2	66	B	82	R	98	b	114	r
3	ETX	19	DC3	35	#	51	3	67	C	83	S	99	c	115	s
4	EOT	20	DC4	36	$	52	4	68	D	84	T	100	d	116	t
5	ENQ	21	NAK	37	%	53	5	69	E	85	U	101	e	117	u
6	ACK	22	SYN	38	&	54	6	70	F	86	V	102	f	118	v
7	BEL	23	ETB	39	'	55	7	71	G	87	W	103	g	119	w
8	BS	24	CAN	40	(	56	8	72	H	88	X	104	h	120	x
9	HT	25	EM	41	)	57	9	73	I	89	Y	105	i	121	y
10	LF	26	SUB	42	*	58	:	74	J	90	Z	106	j	122	z
11	VT	27	ESC	43	+	59	;	75	K	91	[	107	k	123	{
12	FF	28	FS	44	,	60	<	76	L	92	\	108	l	124	|
13	CR	29	GS	45	-	61	=	77	M	93	]	109	m	125	}
14	SO	30	RS	46	.	62	>	78	N	94	^	110	n	126	~
15	SI	31	US	47	/	63	?	79	O	95	_	111	o	127	DEL

Decimal ASCII Chart

https://asciichart.com

Sztringek

A C standard könyvtárának felhasználóbarát leírása:

CS50 Programmer's Manual

Manual pages for the C standard library (and the CS50 Library), with student-friendly annotations.

upper

frequently used in CS50

ctype.h

isupper - character classification functions
tolower - convert uppercase or lowercase
toupper - convert uppercase or lowercase

https://man.cs50.io

Sztringek

Parancssori argumentumok

```
#include <stdio.h>

int main()
{
    printf("hello\n");

    return 0;
}
```

Sztringek

Parancssori argumentumok

```
#include "prog1.h"
#include <stdio.h>

int main(int argc, string argv[])
{
    printf("hello\n");

    return 0;
}
```

Sztringek

Parancssori argumentumok

```
$ ./a.out  hello  world
```

Sztringek

Parancssori argumentumok

```
#include <stdio.h>

int main()
{
    printf("hello\n");

    return 0;
}
```

ha nincs szükségem
a parancssori argumentumokra

```
#include "prog1.h"
#include <stdio.h>

int main(int argc, string argv[])
{
    printf("hello\n");

    return 0;
}
```

ha szükségem van
a parancssori argumentumokra

Kilépési kód

```
#include <stdio.h>

int main()
{
    printf("hello\n");

    return 0;
}
```

- 0 kilépési kód (exit code): minden rendben volt, a program rendesen befejezte a működését
- 0-tól eltérő kilépési kód: valamilyen hiba történt

Kilépési kód

```
1  #include "prog1.h"
2  #include <stdio.h>
3
4  int main(int argc, string argv[])
5  {
6      if (argc != 2)
7      {
8          printf("Hiba! Adj meg egy paramétert!\n");
9          return 1;
10     }
11
12     printf("hello %s\n", argv[1]);
13
14     return 0;
15 }
```

A 9. sorban egy `exit(1);` hívás szebb lenne.
Az `exit()` függvény az `stdlib.h` -ban található meg.

Házi feladat

- A K & R-féle „C Bibliában” nézzék át azokat a részeket, amikről szó volt az előadáson.
- Juhász István jegyzetéből nézzék át azokat a fogalmakat, amikről szó volt az előadáson (link).

- C Style Guide tanulmányozása (link)
- Mi a *linter*? (link, wikipedia)
- A jövő heti gyakorlatra megírt forráskódjainkat nézzük át a style50 programmal is. A programokat próbáljuk megszépíteni.
- A CS50 manual-ban olvassunk utána az `exit()` függvénynek.

Szorgalmi

- Elolvasni: Bug (link, wikipedia)
- Elolvasni: „Le style est l'homme même” (link, wikipedia)